

CONTROLLI PRINCIPALI

IN C# WINDOWS FORM



TEXTBOX

- **Proprietà principali:**

- `Text` → contiene il contenuto della casella.
`nomeCasella.Text = "stringa";`
- `Visible` → booleano, `true` se visibile, `false` se nascosto.

- **Metodi principali:**

- `AppendText("testo")` → aggiunge del testo alla fine di quello già presente.
- `Copy()` → copia il testo selezionato negli appunti.
- `Cut()` → taglia il testo selezionato negli appunti.
- `Paste()` → incolla negli appunti il testo copiato o tagliato.
- `Focus()` → sposta il cursore su questa TextBox, rendendola attiva.

- **Eventi principali:**

- Gli eventi servono a far fare qualcosa alla TextBox quando succede qualcosa.
- `TextChanged` → si attiva quando il testo cambia.
- `KeyDown` → si attiva quando premi un tasto.
- `KeyUp` → si attiva quando rilasci un tasto.
- `KeyPress` → si attiva quando premi e rilasci un tasto.

- **Gestione tasti speciali:**

Quando premi un tasto, puoi anche controllare se premi SHIFT, CTRL o ALT insieme. Tutte queste informazioni si trovano nel parametro **EventArgs** o **KeyPressEventArgs**.

Esempio: *vogliamo controllare se premi SHIFT + F:*

```
private void textBox1_KeyDown(object sender, EventArgs e){  
    if (e.KeyCode == Keys.F && e.Shift == true) {  
        // qui scrivi cosa vuoi far fare  
    }  
}
```

`e.KeyCode == Keys.F` → significa che è stato premuto il tasto **F**.

`e.Shift == true` → significa che il tasto **SHIFT** era premuto insieme.

CHECKBOX / RADIOBUTTON

- **Proprietà principale:**
Checked → true se selezionato, false altrimenti.
- **Differenze:**
 - RadioButton → uno per gruppo.
 - CheckBox → più selezionabili.
- **Gestione gruppi:**
 - Usare Panel o GroupBox per raggruppare RadioButton.
- **Evento tipico:** CheckedChanged, che corrisponde alla loro selezione.

PANEL - GROUPBOX

- Controllo contenitore per raggruppare altri oggetti.
- Nascondendo il Panel si nascondono anche i controlli al suo interno.
- **Inserimento controlli** → trascinare dentro il Panel.
- **Eliminazione Panel** → elimina anche i controlli figli.

GROUPBOX

- Simile al Panel ma con **bordo visibile** a runtime.
- Evidenzia un'area delimitata per i controlli.

PICTUREBOX

Serve a visualizzare immagini (BMP, PNG, GIF anche animate).

Proprietà principali:

- **Image** → contiene il percorso dell'immagine da caricare
- **SizeMode** → regola le proporzioni
 - i. **Normal** è l'impostazione predefinita;
 - ii. **StretchImage** adatta le dimensioni dell'immagine perché sia interamente visualizzata all'interno del controllo;
 - iii. **Autosize** ridimensiona la PictureBox sulla base della grandezza dell'immagine;
 - iv. **CenterImage** visualizza l'immagine centrata nella PictureBox.
- **Esempio codice:**
PictureBox1.Image = System.Drawing.Bitmap.FromFile("C:\\Immagini\\Cascade.bmp");
- Eliminare immagine → **PictureBox1.Image** = null.

LISTBOX

- Mostra un elenco di elementi (di solito stringhe); se l'elenco supera le dimensioni dell'oggetto una barra di scorrimento con le classiche freccette su/giù permetterà di scorrerlo.
- **Proprietà principali:**
 - **Items** → aggiungere/rimuovere elementi:
ListBox1.Items.Add("Nuovo elemento")
ListBox1.Items.Remove("Testing")
ListBox1.Items.RemoveAt(5)
ListBox1.Items.Clear()

LISTBOX

- **Proprietà principali:**
 - `SelectedItem` → elemento selezionato.
 - `SelectedIndex` → indice dell'elemento selezionato, -1 se non è selezionato nulla
 - `Contains(item)` → verifica se un elemento esiste nella lista.
- **Selezione multipla:**
 - `SelectionMode` → `MultipleSimple` o `MultipleExtended`.
- **Evento principale:** `SelectedIndexChanged`, che viene generato quando si seleziona un elemento nella lista, ovvero quando si modifica la proprietà `SelectedIndex` del controllo.

COMBOBOX

- Combina `TextBox` + `ListBox`.
- Permette di scrivere anche voci non presenti nell'elenco.
- Proprietà e metodi simili a `ListBox`.
- Evento principale: `SelectedIndexChanged`.

PER POPOLARLO da CODICE

Usare `Items.AddRange`

```
comboBox.Items.Add("Opzione 1");  
comboBox.Items.Add("Opzione 2");  
comboBox.Items.Add("Opzione 3");
```

Aggiungi uno per
uno gli elementi:

Usare `Items.AddRange`

Se vuoi collegarlo a un **enum** o a una **lista** già pronta: =>

```
comboBox.DataSource = Enum.GetValues(typeof(Razza));
```

oppure con una lista di numeri o stringhe:

```
=> comboBox.DataSource = new List<string>
```

```
{ "Principiante", "Intermedio", "Esperto" };
```